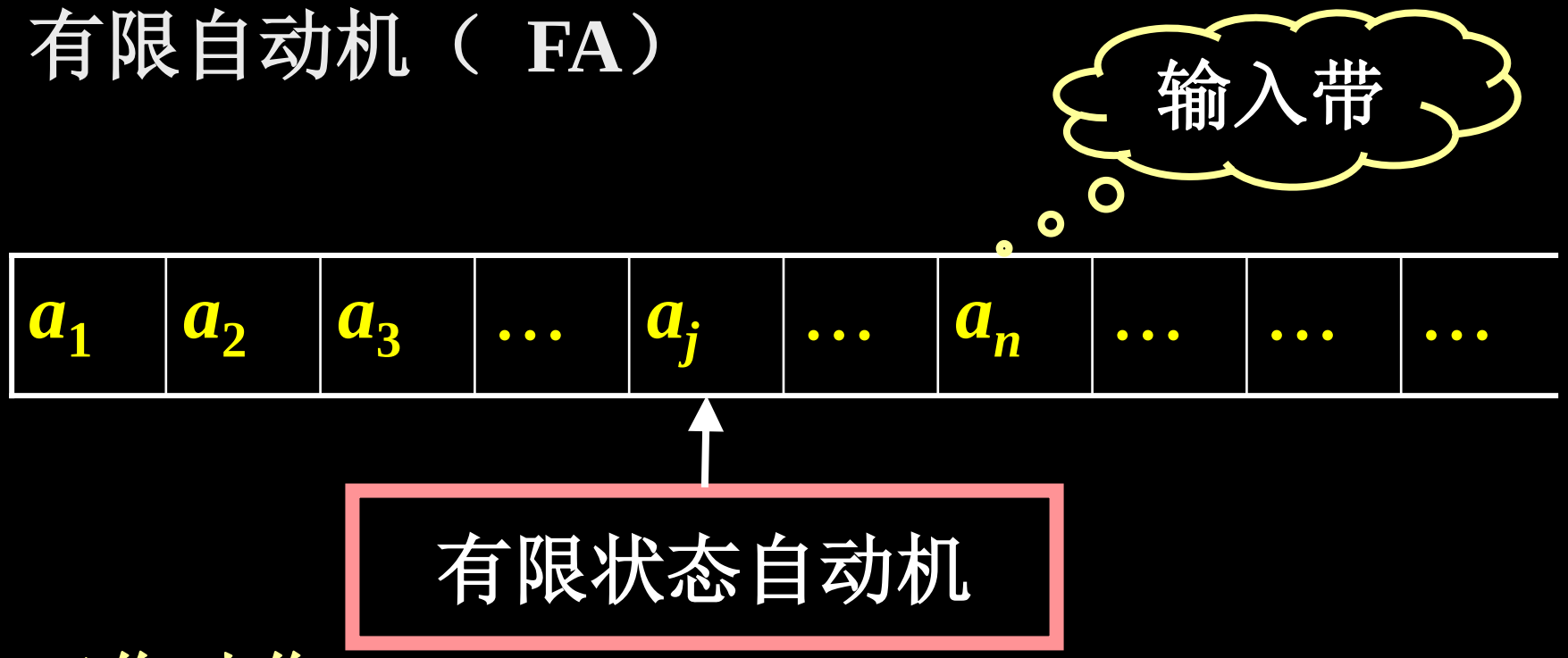
第 2 章 形式语言与自动机基础

2.1 文法和语言

2.2 有限自动机

2.3 正规式与有限自动机

有限自动机（ FA）



工作动作：

读读头所指向的字符；

据当前状态和读的字符改变自动机的状态；

将读头向前移动。

第 2 章　形式语言与自动机基础

2.2 有限自动机基础

2.2.1　确定的有限状态自动机(DFA)

2.2.2　非确定的有限状态自动机(NFA)

2.2.3　NFA确定化

2.2.4　DFA化简

一. 确定的有限自动机（ DFA）

(DFA：Deterministic Finite Automaton)

- **定义2.24**

一个确定的有限自动机M (DFA M)是一个五元组

$$M = (Q, \Sigma, f, q_0, Z)$$

其中：

Q：状态的有限集合，每个元素q_i ($q_i \in Q$) 称为一个状态。

Σ：输入字符的有限集合(或有穷字母表)。

每个元素是一个输入字符。

f：状态转换函数：从$Q\times\Sigma\rightarrow Q$的映射。

例如，$f(p, a)=q$，q、$p\in Q$，$a\in\Sigma$。表示了在状态p读入字符a后转入状态q。q也称为p的后继状态。

q_0：M的唯一初态（也称开始状态），$q_0\in Q$。

Z：M的终态集（或接受状态集）　　$Z\subseteq Q$。

二. DFA的等价表示

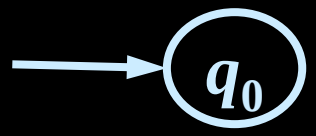
DFA形式定义 ⟷ 状态转换图 ⟷ 状态转换表

例2.20:

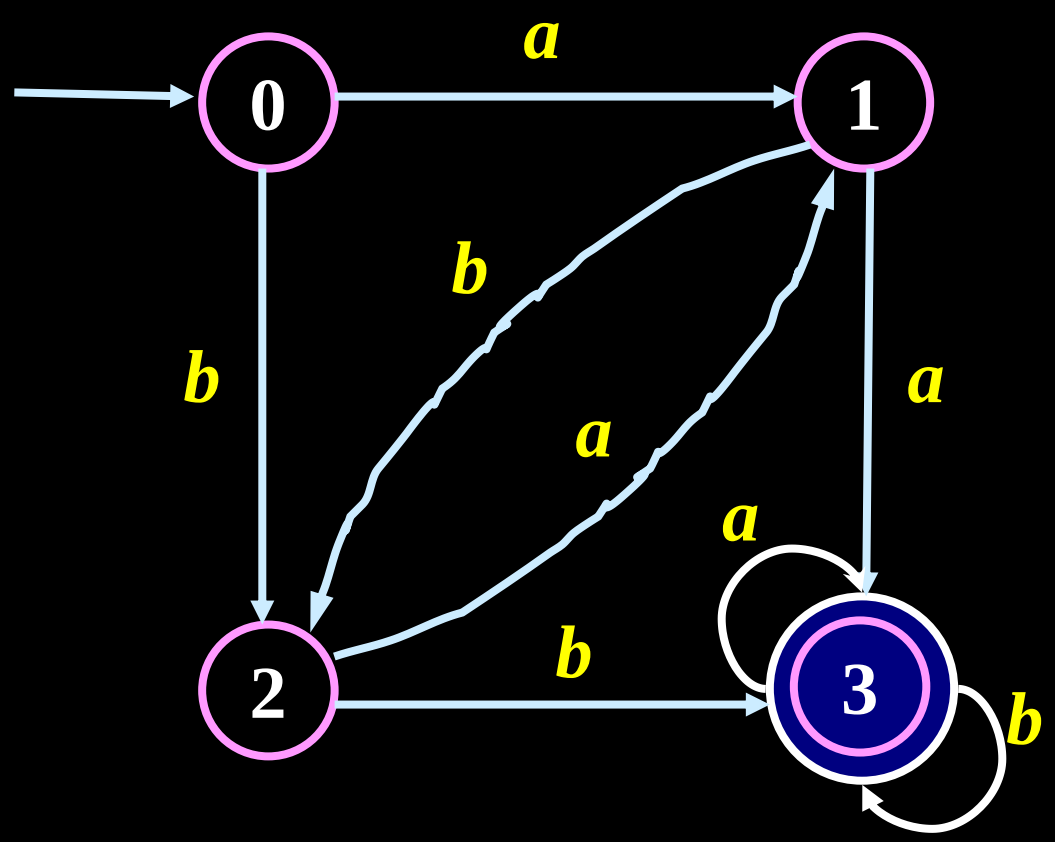
DFA $M = (\underbrace{\{0,1,2,3\}}_{Q}, \underbrace{\{a,b\}}_{\Sigma}, f, \underbrace{0}_{q_0}, \underbrace{\{3\}}_{Z})$

$$\left.\begin{array}{ll} f(0,a)=1 & f(0,b)=2 \\ f(1,a)=3 & f(1,b)=2 \\ f(2,a)=1 & f(2,b)=3 \\ f(3,a)=3 & f(3,b)=3 \end{array}\right\} f$$

状态转换图表示

DFA	状态图
Q	状态结点
Σ	弧上标记(边权值)
f	带权值的有向边
q_0	
Z	

状态转换图表示



DFA M =({0,1,2,3}, {a,b}, f , 0 , {3}）

f（0,a）=1

f（0,b）=2

f（1,a）=3

f（1,b）=2

f（2,a）=1

f（2,b）=3

f（3,a）=3

f（3,b）=3

二. DFA的等价表示

DFA形式定义 ⟷ 状态转换图 ⟷ 状态转换表

例2.20:

$$\text{DFA } M = (\underbrace{\{0,1,2,3\}}_{Q}, \underbrace{\{a,b\}}_{\Sigma}, f, \underbrace{0}_{q_0}, \underbrace{\{3\}}_{Z})$$

$$\left.\begin{array}{ll} f(0,a)=1 & f(0,b)=2 \\ f(1,a)=3 & f(1,b)=2 \\ f(2,a)=1 & f(2,b)=3 \\ f(3,a)=3 & f(3,b)=3 \end{array}\right\} \text{f}$$

- 状态转换表表示

Σ

	a	b
0	1	2
1	3	2
2	1	2
3*	3	3

q_0

Q

Z

f

DFA M =({0,1,2,3}, {a,b}, f , 0 , {3}）

f（0,a）=1

f（0,b）=2

f（1,a）=3

f（1,b）=2

f（2,a）=1

f（2,b）=3

f（3,a）=3

f（3,b）=3

三．DFA的识别机制

确定的有限自动机$M=(Q, \Sigma, f, q_0, Z)$接受或识别字母表Σ上的字符串$\omega=w_1w_2\ldots w_n$的意义：

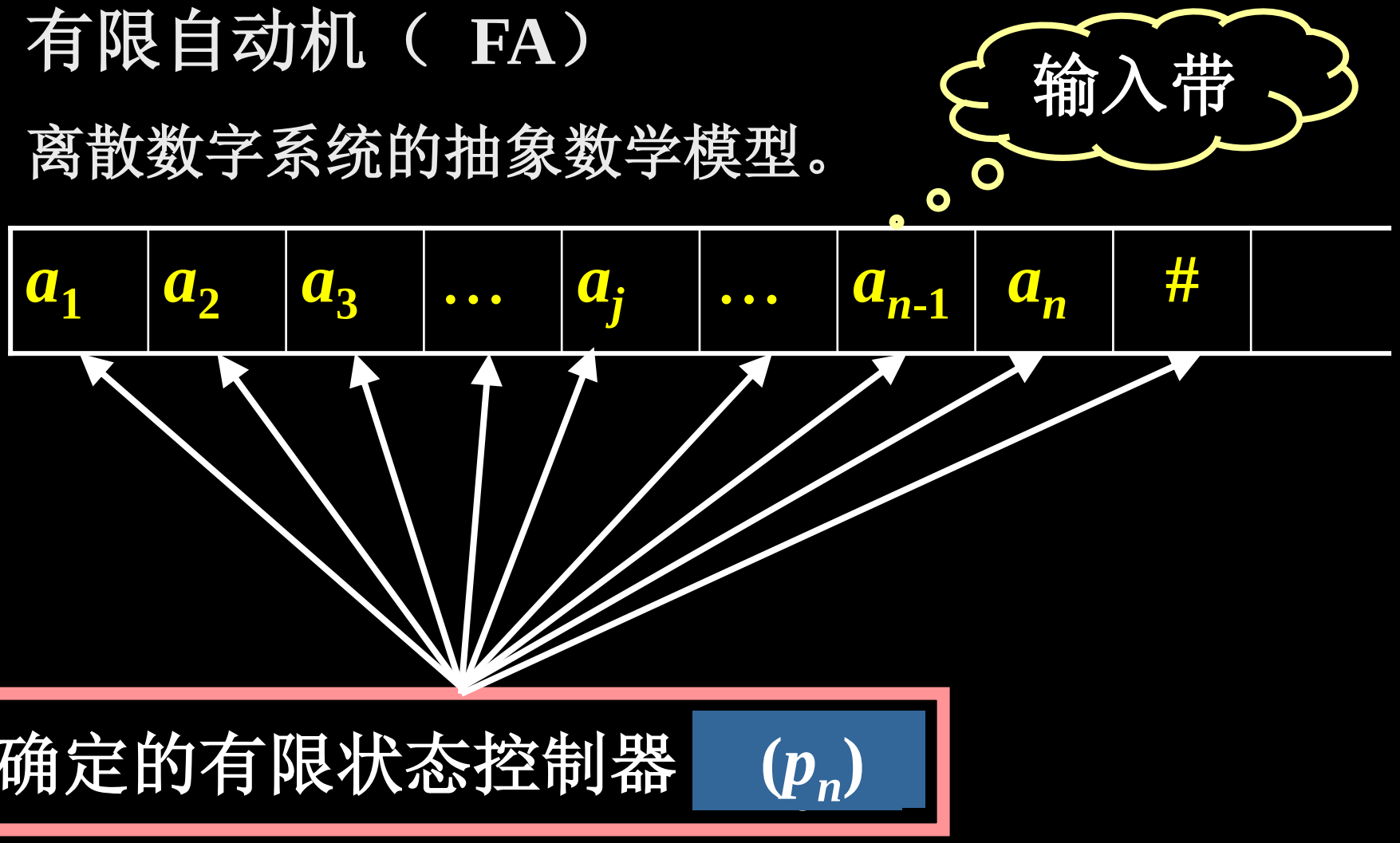
如果存在Q中的状态序列p_0，p_1，…，p_n，满足下列条件：

1) $p_0=q_0$
2) $f(p_i, w_{i+1})=p_{i+1}$，$i=0, 1, \ldots, n-1$
3) $p_n \in Z$

则M接受（识别）ω。

有限自动机（ FA）

离散数字系统的抽象数学模型。



具体识别方法：

根据串沿着序列（路径）p_0，p_1，…，找到p_n，判断p_n是否属于终态集。

如果存在Q中的状态序列p_0，p_1，…，p_n，满足下列条件：

1) $p_0=q_0$

2) $f(p_i, w_{i+1})=p_{i+1}$，$i=0, 1, \ldots, n-1$

3) $p_n \in Z$

则M接受（识别）ω。

寻找

判断

从状态图出发可以更形象地进行描述。

一条路径；开始结点为初态结点；弧上的标记连接成字符串ω；路径终点为终态结点

则称ω被该DFA M所识别(接受)。

特例的是，若M的初态结点同时又是终态结

设$\alpha = a_1 a_2 \ldots\ldots a_{n-1} a_n$，

$f(q_0,\alpha)=f(f(\ldots f(f(q_0,a_1,),a_2),\ldots,a_{n-1}),a_n)$

为M识别的语言，记为$L(M)$。

$$L(M) = \{\alpha \mid \alpha \in \Sigma^* \wedge f(q_0, \alpha) \in Z\}$$

例2.21:

分析下面描述的DFA M_1。

DFA $M_1=(\{q_{ee}, q_{oe}, q_{eo}, q_{oo}\}, \{0,1\}, f, q_{ee}, \{q_{ee}\})$

其中：

$f(q_{ee}, 0)= q_{oe}$　　$f(q_{ee}, 1)= q_{eo}$

$f(q_{oe}, 0)= q_{ee}$　　$f(q_{oe}, 1)= q_{oo}$

$f(q_{eo}, 0)= q_{oo}$　　$f(q_{eo}, 1)= q_{ee}$

$f(q_{oo}, 0)= q_{eo}$　　$f(q_{oo}, 1)= q_{oe}$

- $1=110101，$M_1$对$1的计算过程是：

$f(q_{ee},1)=q_{eo}$

$f(q_{eo},1)=q_{ee}$

$f(q_{ee},0)=q_{oe}$

$f(q_{oe},1)=q_{oo}$

$f(q_{oo},0)=q_{eo}$

$f(q_{eo},1)=q_{ee}\in Z$

所以串$1= 110101可以被$M_1$接受。

$f(q_{ee},110101)=f(f(q_{ee},1),10101)=\ldots\ldots$

$=f(q_{eo},1)=q_{ee}\in Z$

$(\{q_{ee},q_{oe},q_{eo},q_{oo}\},\{0,1\},f,q_{ee},\{q_{ee}\})$

$f(q_{ee},0)=q_{oe}$

$f(q_{ee},1)=q_{eo}$

$f(q_{oe},0)=q_{ee}$

$f(q_{oe},1)=q_{oo}$

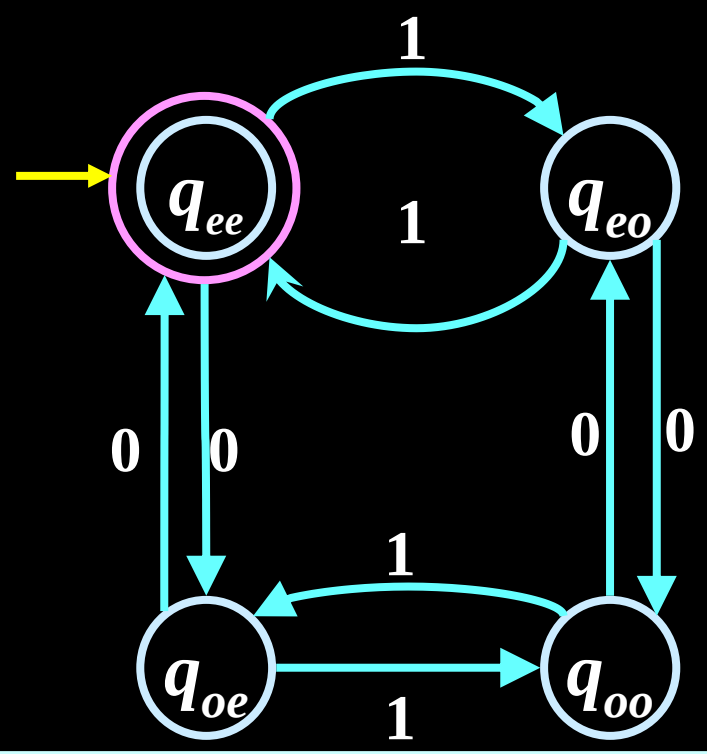
$f(q_{eo},0)=q_{oo}$

$f(q_{eo},1)=q_{ee}$

$f(q_{oo},0)=q_{eo}$

$f(q_{oo},1)=q_{oe}$

DFA M_1状态图



可以识别的语言为含偶数个0和偶数个1的二进制串集合。

对1010:

$$q_{ee} \xrightarrow{1} q_{eo} \xrightarrow{0} q_{oo} \xrightarrow{1} q_{oe} \xrightarrow{0} q_{ee} \in Z$$

所以串1010被该DFA接受！

对11001:

$$q_{ee} \xrightarrow{1} q_{eo} \xrightarrow{1} q_{ee} \xrightarrow{0} q_{oe} \xrightarrow{0} q_{ee} \xrightarrow{1} q_{eo}$$

所以串11001被该DFA拒绝！

例2.22：设计一台DFA，接受含有子串001的所有二进制串。

- **问题分析：**

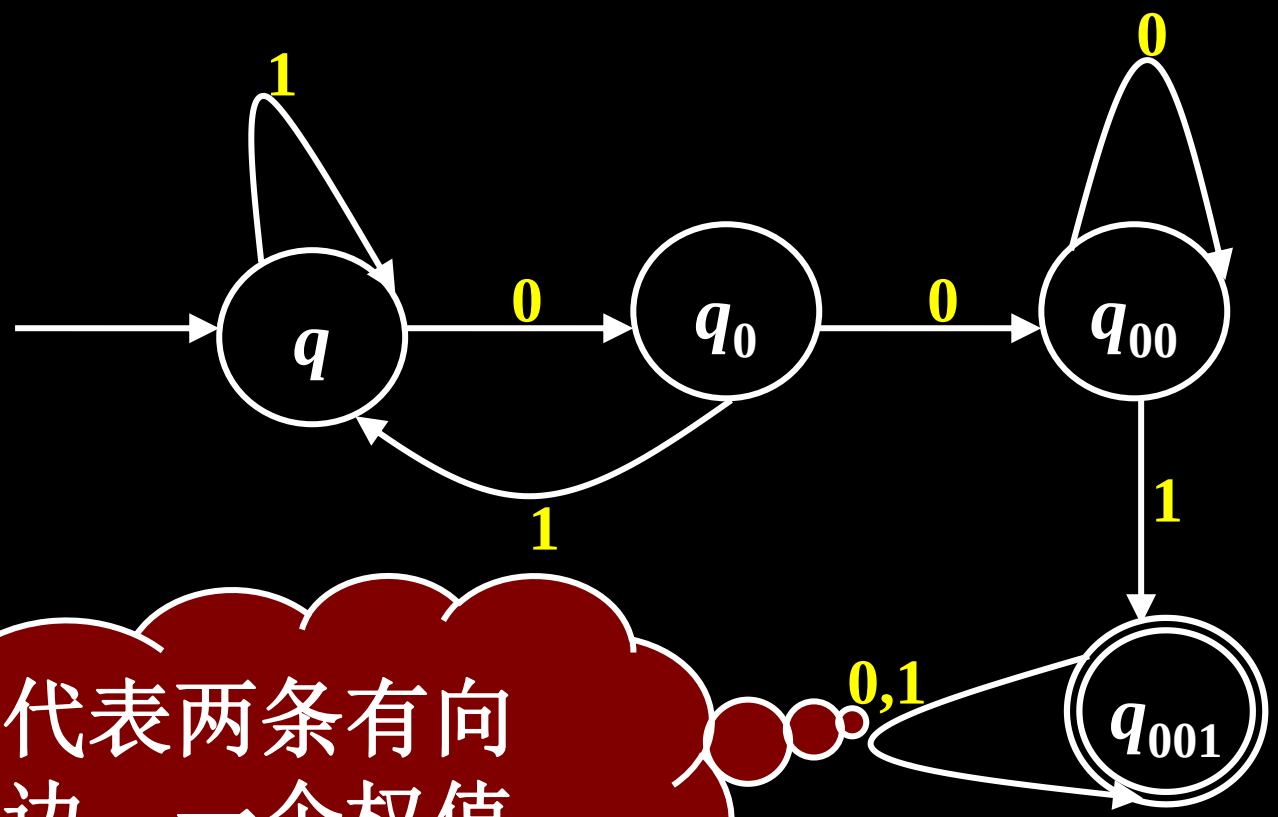
输入字母为0或1，所以$\Sigma=\{0,1\}$

扫描过程中有4种可能性：

1) 开始寻找001；
2) 读到一个0；
3) 读到了00；
4) 读到了001

所以有4个状态$Q=\{q, q_0, q_{00}, q_{001}\}$,其中$q$为初态，$q_{001}$为终态。

接受含有子串001的所有二进制串的DFA



代表两条有向边，一个权值为0，一个为1

与文法等价概念类似：

设有DFA M 和DFA M'， 若$L(M) = L(M')$，则称M 和 M' 等价。

✌ 注意：

1） DFA是具有离散输入、输出系统的一个纯数学模型；

2） DFA的技巧在于状态的设置；

3） DFA映射的唯一性。（对于任意字，在DFA中有且仅有唯一路径）。

第 2 章　形式语言与自动机基础

2.2 有限自动机基础

2.2.1　确定的有限状态自动机(DFA)

2.2.2　非确定的有限状态自动机(NFA)

2.2.3　NFA确定化

2.2.4　DFA化简

一. NFA的定义

DFA的确定性表现在其映射函数是一个单值函数。但是实际问题中，映射函数往往是一个多值函数。

例如，源程序中扫描到一个字母时，不同的语言对应多种情况：

FORTRAN中： 标识符/格式转换码E、D…

C语言中：标识符/ if / switch ….

例如，

C语言中：{ +　++　+= }

∴ NFA在实际中更具普遍性。

- **定义2.25**

一个非确定的有限自动机M (NFA M)是一个五元组

$$M = (Q, \Sigma, f, q_0, Z)$$

其中：

Q， Σ， Z， q_0同DFA。

f：状态转换函数。

从$Q\times(\Sigma\cup\{\varepsilon\})\rightarrow 2^Q$的映射。这里的后继状态不是唯一的，它是状态集$Q$的子集。

✌ 注意：

1) NFA亦可用状态图和状态表表示。

2) DFA和NFA统称为有限自动机FA。

例2.23: 设有一个非确定的有限自动机M

NFA $M=(\underbrace{\{q_0, q_1, q_2, q_3, q_4\}}_{Q},\underbrace{\{0,1\}}_{\Sigma}, f, \underbrace{q_0}_{q_0}, \underbrace{\{q_2,q_4\}}_{Z})$

其中状态转换函数f为：

$f(q_0, 0)= \{q_0,q_3\}$　$f(q_0, 1)= \{q_0, q_1\}$

$f(q_1, 1)=\{q_2\}$

$f(q_2, 0)= \{q_2\}$　$f(q_2, 1)= \{q_2\}$

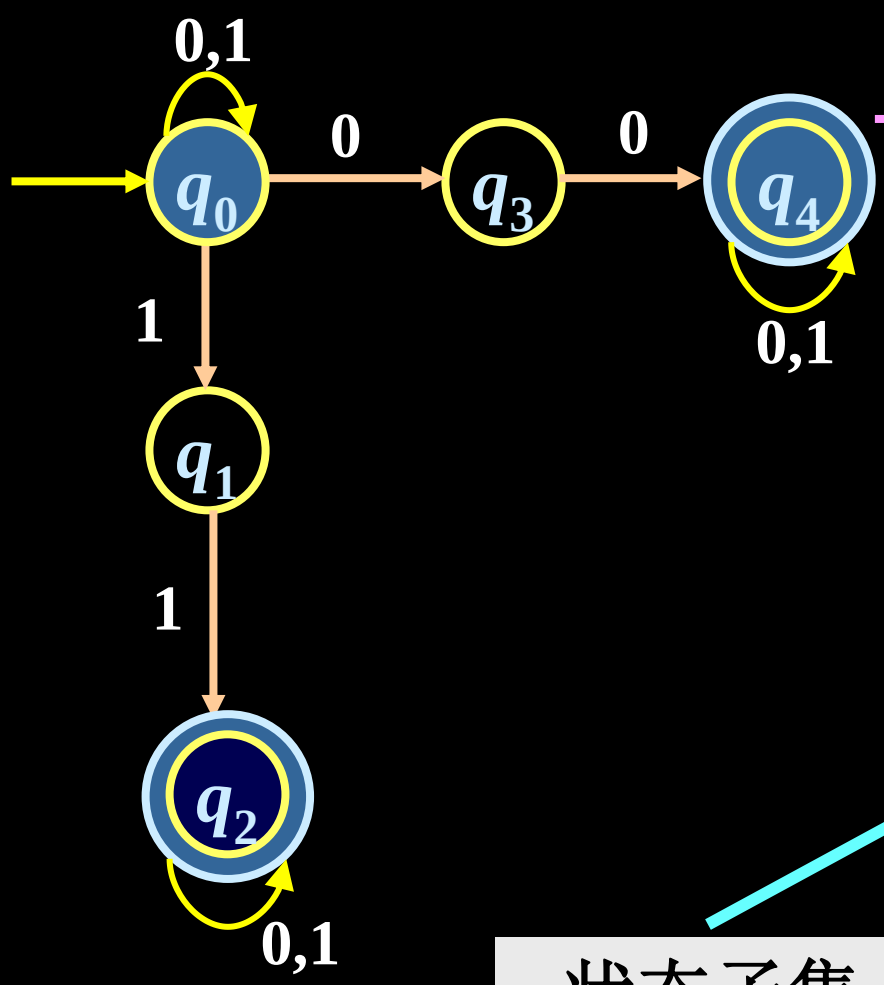
$f(q_3, 0)= \{q_4\}$

$f(q_4, 0)= \{q_4\}$　$f(q_4, 1)=\{q_4\}$

$f(q_0,0)=\{q_0,q_3\}$ $f(q_0,1)=\{q_0,q_1\}$ $f(q_1,0)=\Phi$ $f(q_1,1)=\{q_2\}$

$f(q_2,0)=\{q_2\}$ $f(q_2,1)=\{q_2\}$ $f(q_3,0)=\{q_4\}$ $f(q_3,1)=\Phi$

$f(q_4,0)=\{q_4\}$ $f(q_4,1)=\{q_4\}$



	0	1
q_0	$\{q_0,q_3\}$	$\{q_0,q_1\}$
q_1		$\{q_2\}$
q_2*	$\{q_2\}$	$\{q_2\}$
q_3	$\{q_4\}$	
q_4*	$\{q_4\}$	$\{q_4\}$

状态子集

二．NFA的识别机制

非确定的有限自动机$M=(Q, \Sigma, f, q_0, Z)$接受或识别字母表Σ上的字符串$\omega=w_1w_2\ldots w_n$，$w_i\in\Sigma\cup\{\varepsilon\}$的意义：

如果存在Q中的状态序列p_0，p_1，…，p_n，满足下列条件：

1) $p_0=q_0$

2) $p_{i+1}\in f(p_i, w_{i+1})$，$i=0, 1, \ldots, n-1$

3) $p_n\in Z$

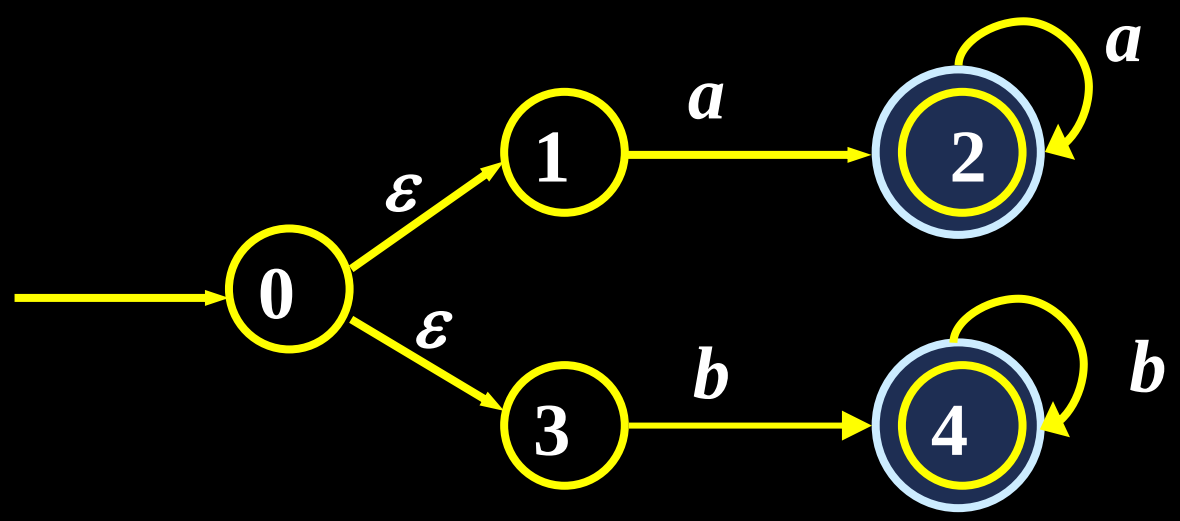
则M接受（识别）ω。

从状态转换图进行描述：

一条路径；开始结点为初态结点；弧上的标记连接成字符串ω；路径终点为终态结点则称ω被该DFA M所识别(接受)。

非确定的有限自动机M识别的字符串的全体称为M识别的语言，记为$L(M)$。

例2.24: 给出一个识别语言为$\{a\}^+ \cup \{b\}^+$的NFA M 如下图所示。

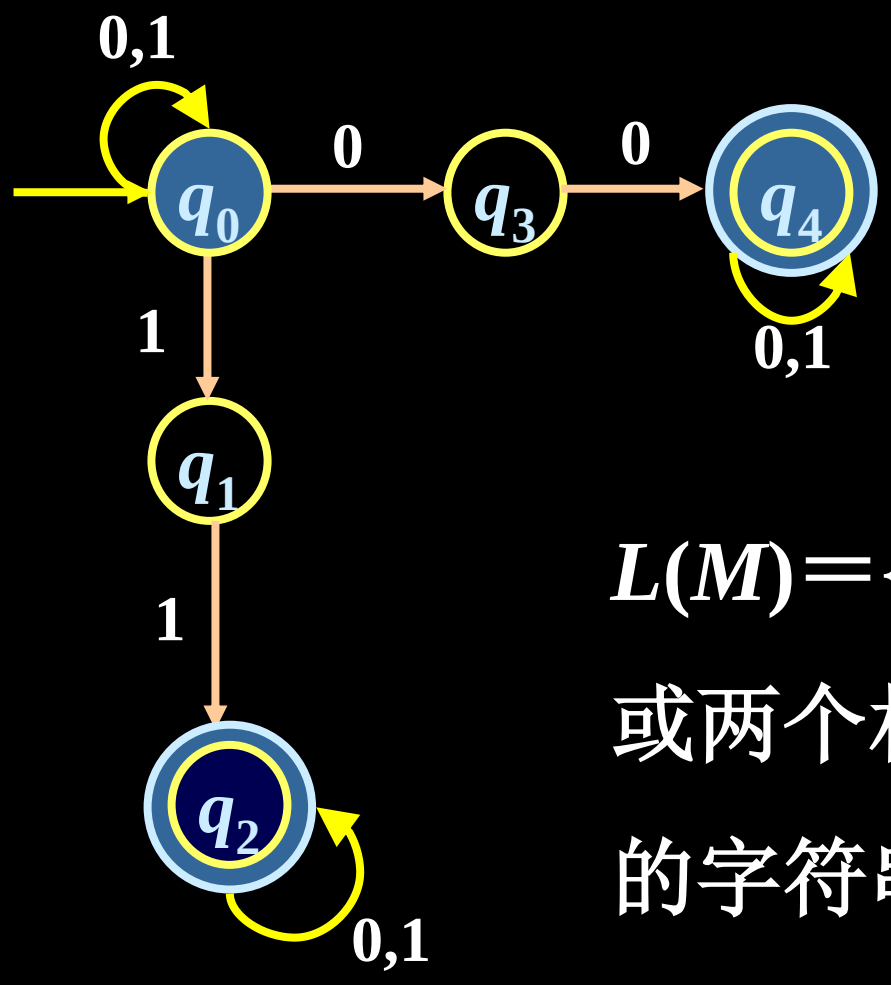


对字符串*aaa*的计算路径：

$$0 \xrightarrow{\varepsilon} 1 \xrightarrow{a} 2 \xrightarrow{a} 2 \xrightarrow{a} 2 \in Z$$

$$0 \xrightarrow{\varepsilon} 3 \times$$

例2.23的非确定的有限自动机M所识别的语言L（M）？



$L(M)$＝{含有两个相邻的0或两个相邻的1的由0和1组成的字符串}

例2.22：设计一台FA，接受含有子串001的所有二进制串。

- **问题分析：**

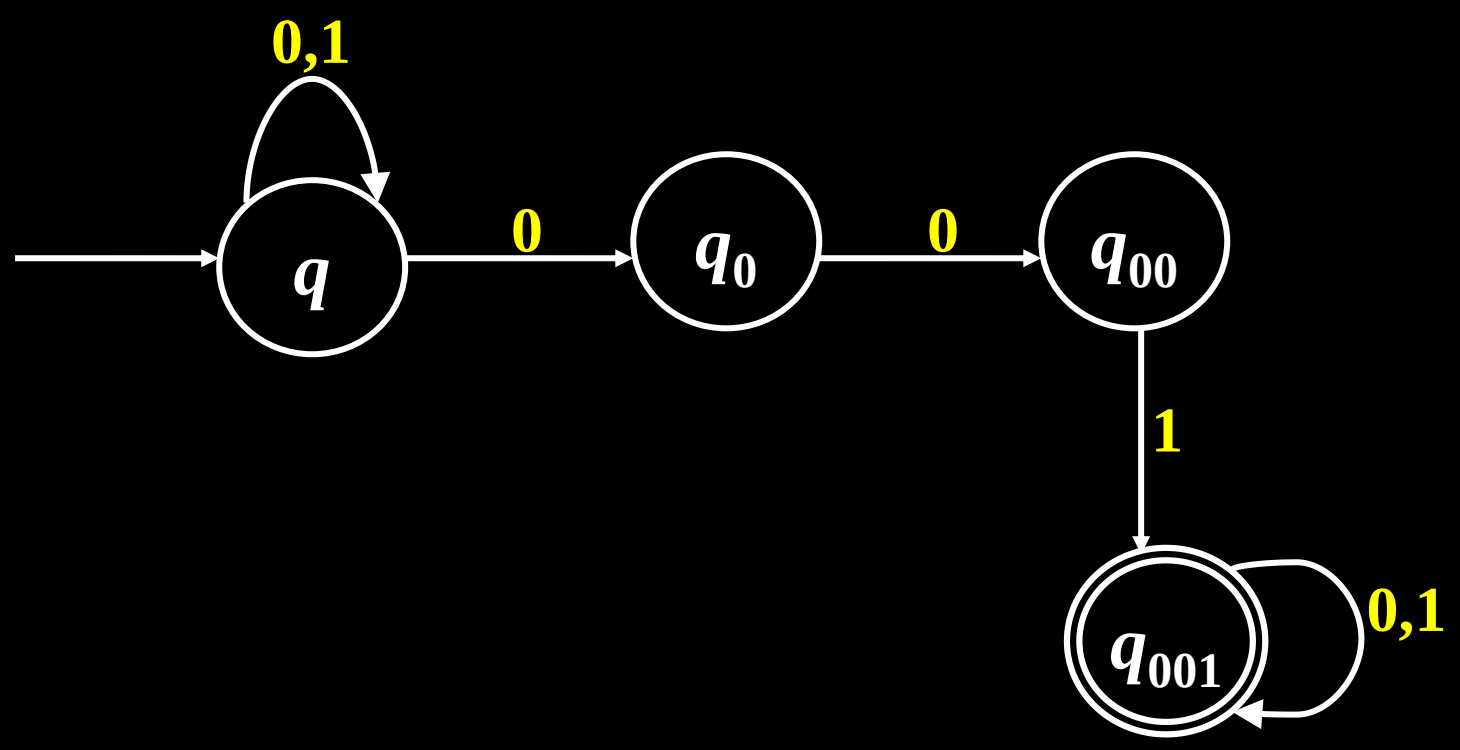
输入字母为0或1，所以$\Sigma=\{0,1\}$

扫描过程中有4种可能性：

1) 开始寻找001；
2) 读到一个0；
3) 读到到00；
4) 读到了001

所以有4个状态$Q=\{q, q_0, q_{00}, q_{001}\}$,其中$q$为初态，$q_{001}$为终态。

接受含有子串001的所有二进制串的FA



三. NFA和DFA的区别

	NFA	DFA
f	$f(Q\times\Sigma\cup\{\varepsilon\})$	$f(Q\times\Sigma)$
	$f(Q\times\Sigma\cup\{\varepsilon\})\longrightarrow 2^Q$	$f(Q\times\Sigma)\longrightarrow Q$

♥ 注意：

在NFA中对字的识别时验证的路径可能不唯一。

第 2 章　形式语言与自动机基础

2.2 有限自动机基础

2.2.1　确定的有限状态自动机(DFA)

2.2.2　非确定的有限状态自动机(NFA)

2.2.3　NFA确定化

2.2.4　DFA化简

- **定理2.1**

对任何一个NFA M，都存在一个DFA M'，使 $L(M')=L(M)$。

定理2.1说明：对任何一个NFA M，都存在一个DFA M'，使M和M'所识别的字的全体相同，我们可简记为 $M'=M$。

- **NFA确定化的算法 — 子集法。**

- **定义2.26**

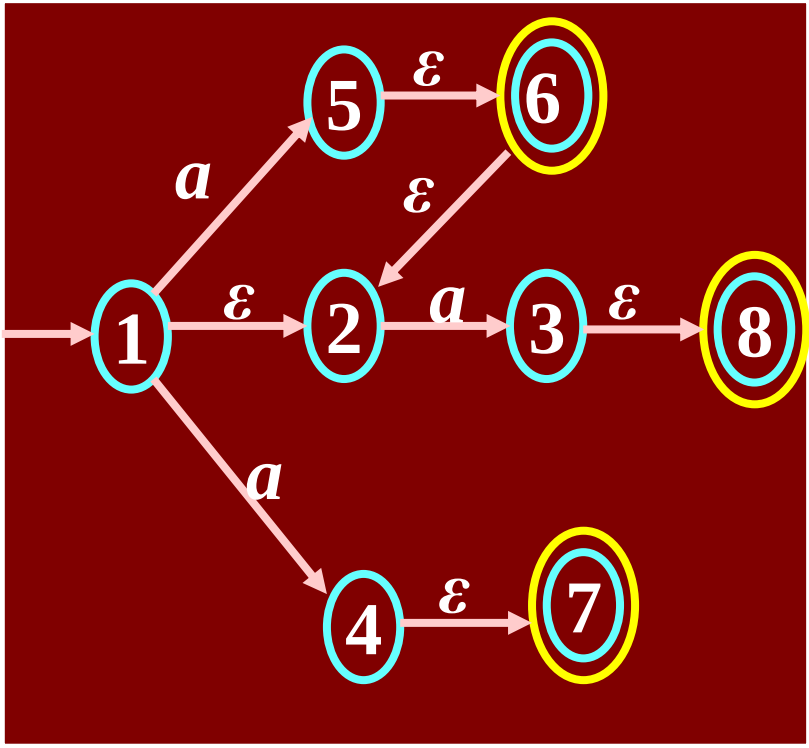
I是NFA M' 状态集Q的一个子集。(即$I \subseteq Q$)，

则ε-closure(I)为：

(1) $q_i \in I$，$q_i \in \varepsilon$-closure(I);

(2) $q_i \in I$，从q_i出发经过任意条ε弧而能到达的任何状态q_j，$q_j \in \varepsilon$-closure(I)。

例2.25:有NFA *M*如右图所示。



$I=\{5\}$

ε-closure(I) = ε-closure({5})

= {5, 6, 2}

I={1}

ε-closure({1})

= {1, 2}

I= {1,5}

ε-closure({1,5})

＝ε-closure ({1}) ∪ ε-closure ({5})

＝{1,2,5,6}

✌ **综述：**

1）状态集I的ε-closure(I)仍是一状态集；

2）状态集（ε-closure(I)）即为在I中的状态下，不输入任何字符所能到达的状态的全体并包含I中的状态，就是状态集I的ε闭包。

- **算法2.1 (求I的ε-closure(I))**

输入： NFA M 和M的子集I

输出： ε-closure(I)

算法：

```
set_of_state  look (set_of_state I)
    {
      look=I;
      do
        {对look中每一个状态i
          if ∃ (i) --ε--> (j) 结构
            look = look + {j};
          }while(look不再扩大)
    }
```

- **定义2.27 （状态集合I的a弧转换I_a ）**

设 NFA $M'=(Q, \sum, f, q_0, Z)$

假定$I \subseteq Q$，$a \in \sum$，则定义

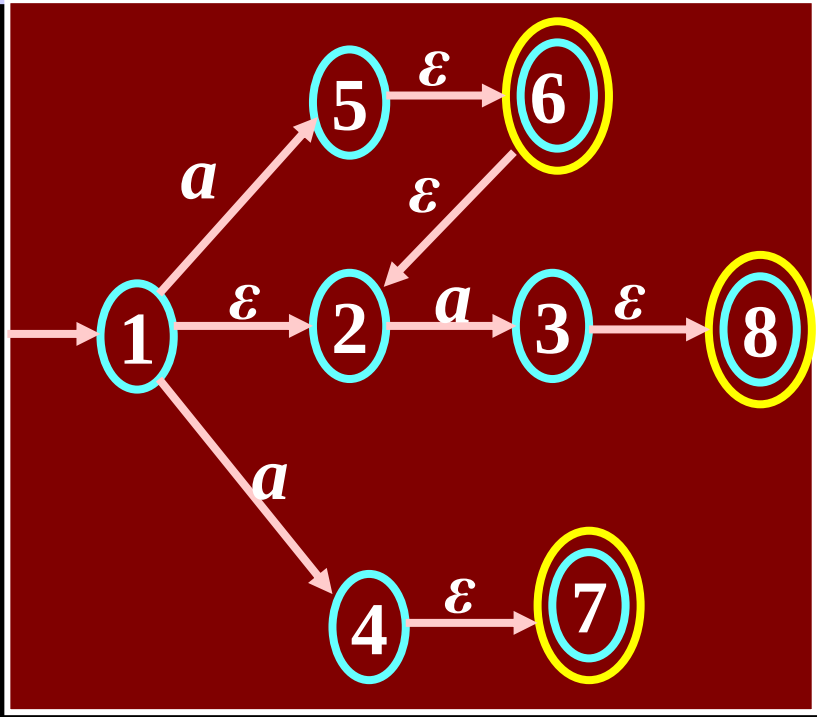
$I_a=\varepsilon\text{-closure}(\{p \mid \exists q \in \varepsilon\text{-close}(I), p \in f(q, a)\})$。

✌ **注意：**

计算I_a需三步：

I的ε闭包；闭包的映射集；映射集的ε闭包。

$I_a=\varepsilon\text{-closure}(f(\varepsilon\text{-close}(I), a))$。



例2.26: 有NFA M如例2.25。

设I={1} 求I_a

则ε-closure（I）＝{1, 2}

$f(\{1,2\},a)=f(1,a)\cup f(2,a)$

$=\{3,4,5\}$

I_a =ε-closure（{3,4,5}）

= { 2, 3, 4, 5, 6,7,8 }

设I={2,5}

则 I_a =ε-closure（$f(\{2,5,6\},a)$）

=ε-closure（{3}）= { 3,8 }

- **NFA确定化关键：**

1） 消去ε弧； ← ε-closure(I)

2） 解决映射不唯一问题。 ← 求I_a

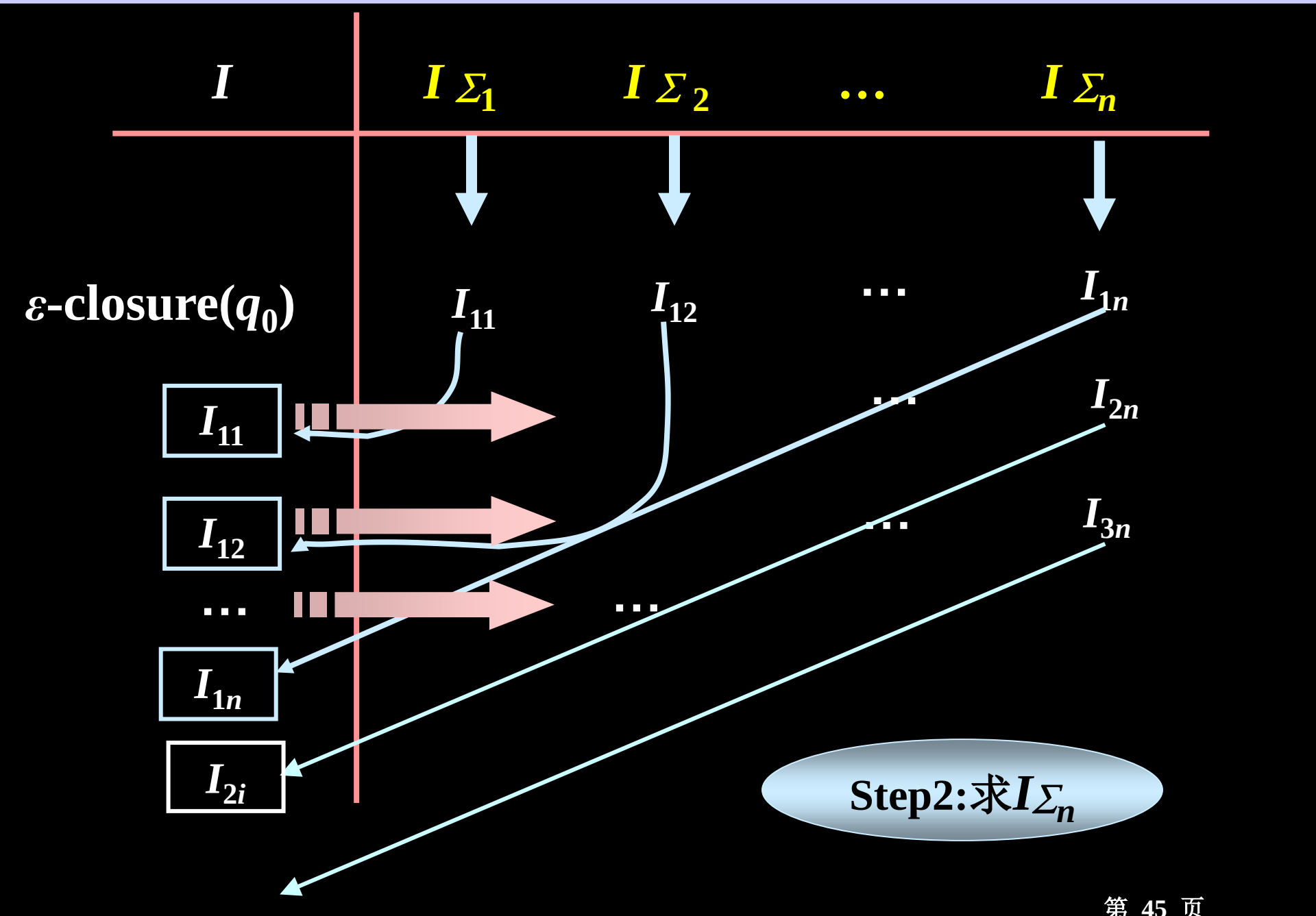
- 子集法——NFA的确定化算法

对NFA $M' = (Q, \{\Sigma_1, \Sigma_2, \dots, \Sigma_n\}, f, q_0, Z)$

设一状态表：

I	$I\Sigma_1$	$I\Sigma_2$	…	$I\Sigma_n$
ε-closure(q_0)				

Step1:初始化


I
$I\Sigma_1$
$I\Sigma_2$
…
$I\Sigma_n$
ε-closure(q_0)
I_{11}
I_{12}
…
I_{1n}
I_{2n}
I_{3n}
I_{11}
I_{12}
…
I_{1n}
I_{2i}
Step2:求$I\Sigma_n$

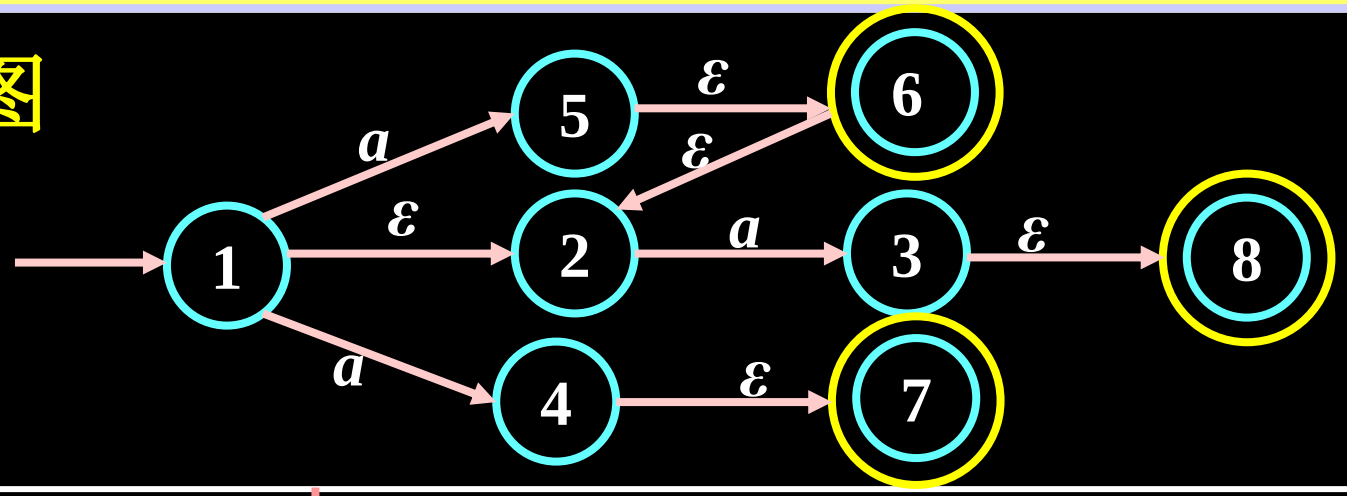
Step3: 重新命名

对求得的状态表（DFA *M*）的第一列各状态子集重新命名为状态，然后代入相应的状态表元素；

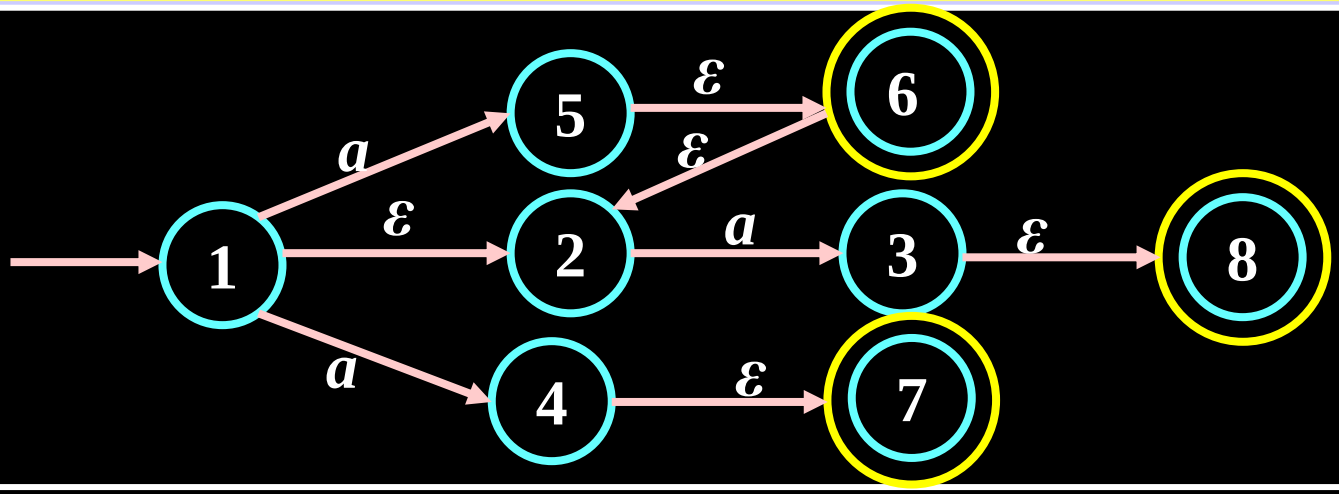
第一列第一行为DFA *M* 的惟一初态；

含有原*M'*终态的*I*为*M*终态。

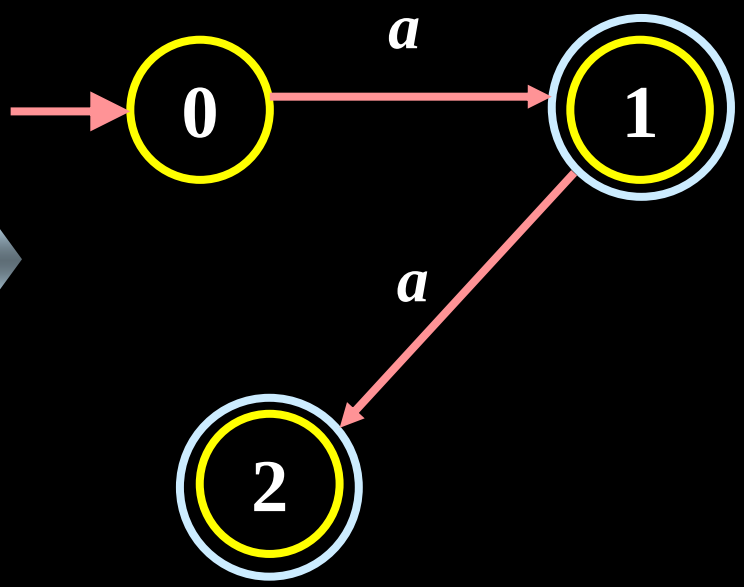
例2.27:将右图所示NFA *M'* 确定化。



	I	I_a	
0	ε-closure(q_0) {1,2}	{2, 3, 4, 5, 6, 7, 8}	1
1*	{2, 3, 4, 5, 6, 7, 8}	{ 3, 8}	2
2*	{ 3, 8}	Φ	

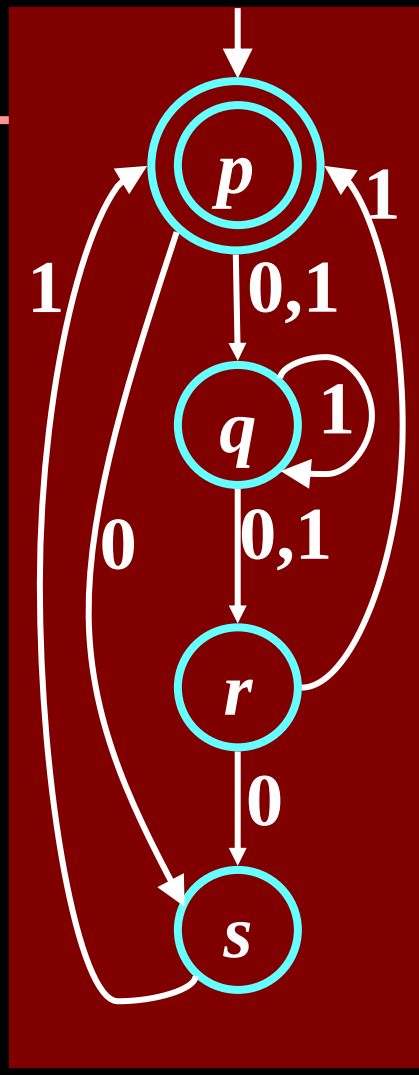


	a
0	1
1*	2
2*	

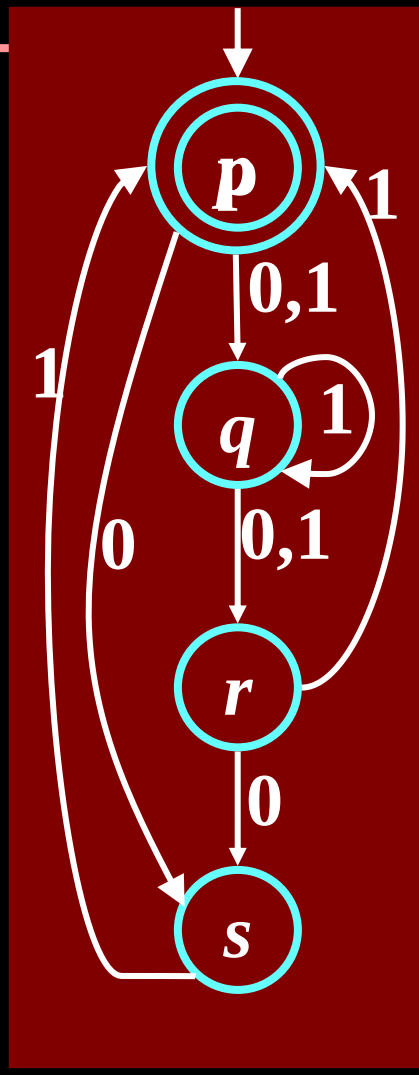


例2.28: 将右下图所示NFA *M'* 确定化。

	I		I_0	I_1	
0*	{ *p* }	1	{*q*, *s*}	{ *q* }	2
1	{*q*, *s*}	3	{ *r* }	{ *q*,*r*,*p* }	4
2	{ *q* }	3	{ *r* }	{ *q*,*r* }	5
3	{ *r* }	6	{ *s* }	{ *p* }	0
4*	{ *q*,*r*,*p* }	7	{ *q*,*r*,*s* }	{ *q*,*r*,*p* }	4
5	{ *q*,*r* }	8	{ *r*,*s* }	{ *q*,*r*,*p* }	4
6	{ *s* }		*Φ*	{ *p* }	0
7	{ *q*,*r*,*s* }	8	{ *r*,*s* }	{ *q*,*r*,*p* }	4
8	{ *r*,*s* }	6	{ *s* }	{ *p* }	0



state	0	1
0*	1	2
1	3	4
2	3	5
3	6	0
4*	7	4
5	8	4
6		0
7	8	4
8	6	0



第 2 章　形式语言与自动机基础

2.2 有限自动机基础

2.2.1　确定的有限状态自动机(DFA)

2.2.2　非确定的有限状态自动机(NFA)

2.2.3　NFA确定化

2.2.4　DFA化简

- 所谓DFA *M*的化简是指寻找一个状态数最少的DFA *M′*（规约或最小的DFA *M′*），使得L(*M*)=L(*M′*)。

  已证明存在一个最小DFA *M′*，使得L(*M*)=L(*M′*)。

- 定义2.28

  如果DFA *M*既没有无关状态，且没有彼此等价的状态，则称DFA *M*是规约的(即最小的DFA *M*)。

- **DFA化简实现思想：**

  通过删除无关状态，合并等价状态的规约过程，直至得到规约机(最小的DFA)。

- **定义2.29**（无关状态或多余状态或无用状态）

  如果从DFA *M*的初态开始，任何输入序列都不能到达的那些状态称为无关状态。

▪删除DFA无关状态的算法：

(1) 标记开始状态q_0；

(2) while（存在未处理的标记状态）

{

取未处理的标记状态q，标记为处理，

对所有$a \in \Sigma$,

若$f(q, a)=p$，且p未标记，标记p；

}

(3) 删除未标记的状态及与其相关的转换。

例2.29：有FA *M*

考察：

0：从状态0开始，经过输入0或1 ⟶ 1，5；

1：从状态1开始，经过输入0或1 ⟶ 2，7；

5：从状态5开始，经过输入0或1 ⟶ 3，1；

2：从状态2开始，经过输入0或1 ⟶ 2，5；

7：从状态7开始，经过输入0或1 ⟶ 0，1；

3：从状态3开始，经过输入0或1 ⟶ 5，7；

state	0	1
0	1	5
1	2	7
2	2	5
3	5	7
~~4~~	~~5~~	~~6~~
5	3	1
~~6~~	~~8~~	~~0~~
7	0	1
~~8~~	~~3~~	~~6~~

从初态0开始，4,6,8是不可到达的状态。

state	0	1
0	1	5
1	2	7
2	2	5
3	5	7
5	3	1
7	0	1

定义2.30 （等价状态、可区分状态）

设DFA M的两个不同状态 q_1，q_2，如果对任意输入字符串ω，从q_1，q_2状态出发，总是同时到达接收状态或拒绝状态之中，称q_1，q_2是等价的。

即对$\forall\omega(\omega\in\sum^*)$，$f(q_1，\omega)= p_1$， $f(q_2，\omega)=p_2$，

p_1 ,$p_2\in Z$ 或p_1 ,$p_2\notin Z$，

则q_1，q_2等价，记作 $q_1\sim q_2$。

如果两个状态不等价，

则称q_1，q_2是可区别的（q_1，q_2 被ω 所区别）。

- **DFA合并等价状态的实现方法：划分法。**

划分法的核心：寻找且合并等价状态。

即：将DFA的状态集划分为互不相交的子集，使得任何两个不同子集的状态都是可区分的，而同一个子集的任何两个状态都是等价的。然后每个子集中的状态合并为一个状态。

- 划分法的算法实现步骤如下：

(1) 把M的所有状态Q按终态与非终态划分成两个状态子集Z及$Q—Z$，构成初始划分(或称基本划分)，记作：$\pi=\{Z, Q—Z\}$；

(2) 设当前的划分 π 中已经含有m个子集，即：

$$\pi=\{Q_1，Q_2，\ldots，Q_m\}$$

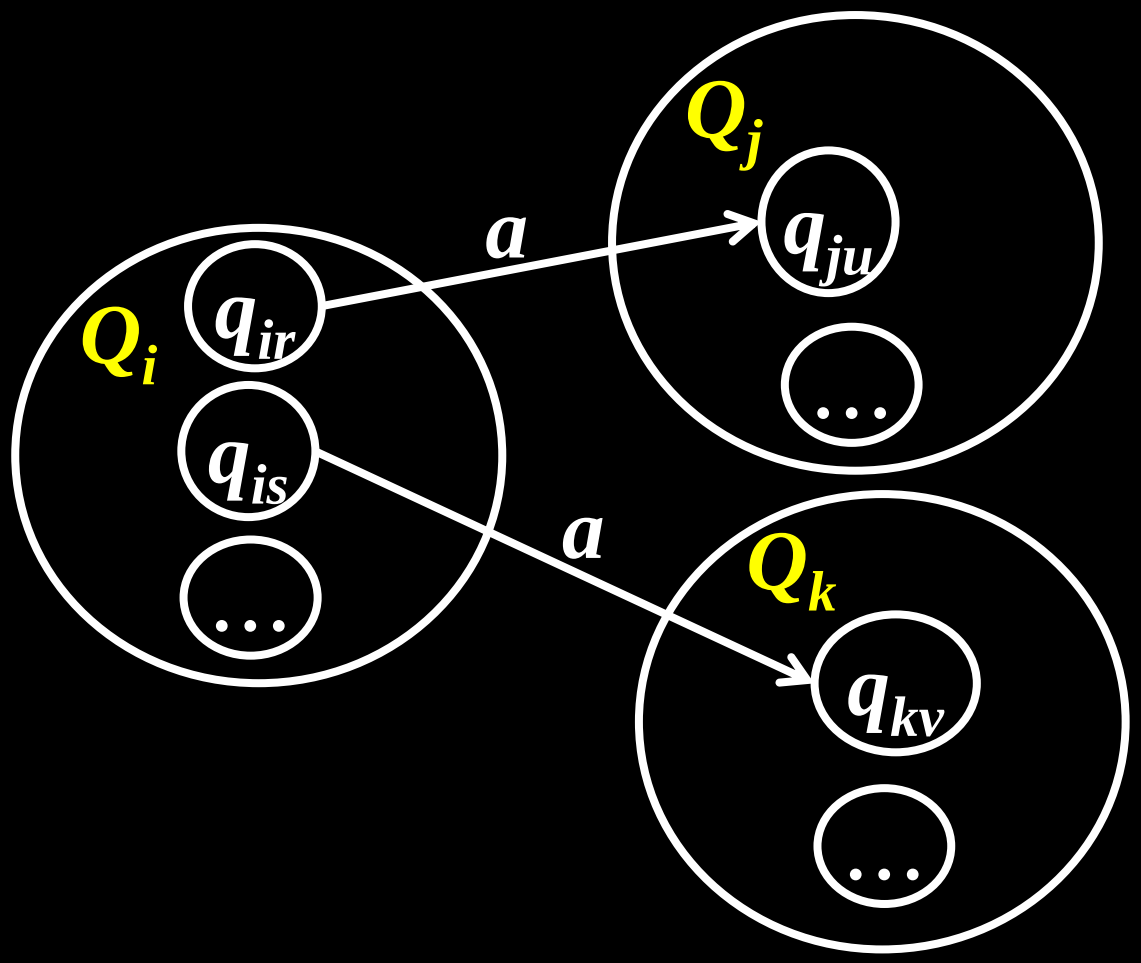
其中，属于不同子集的状态是可区分的，而属于同一子集中的各状态是待区分的。即需要对每一个子集$Q_i=\{q_{i1}，q_{i2}，\ldots，q_{in}\}$中各状态$q_{ir}$ ($q_{ir}\in Q$, $1\le r\le n$) 进行考察，确认是否还能对它们进行划分。

若能进行划分，则形成新的划分π_{new}。

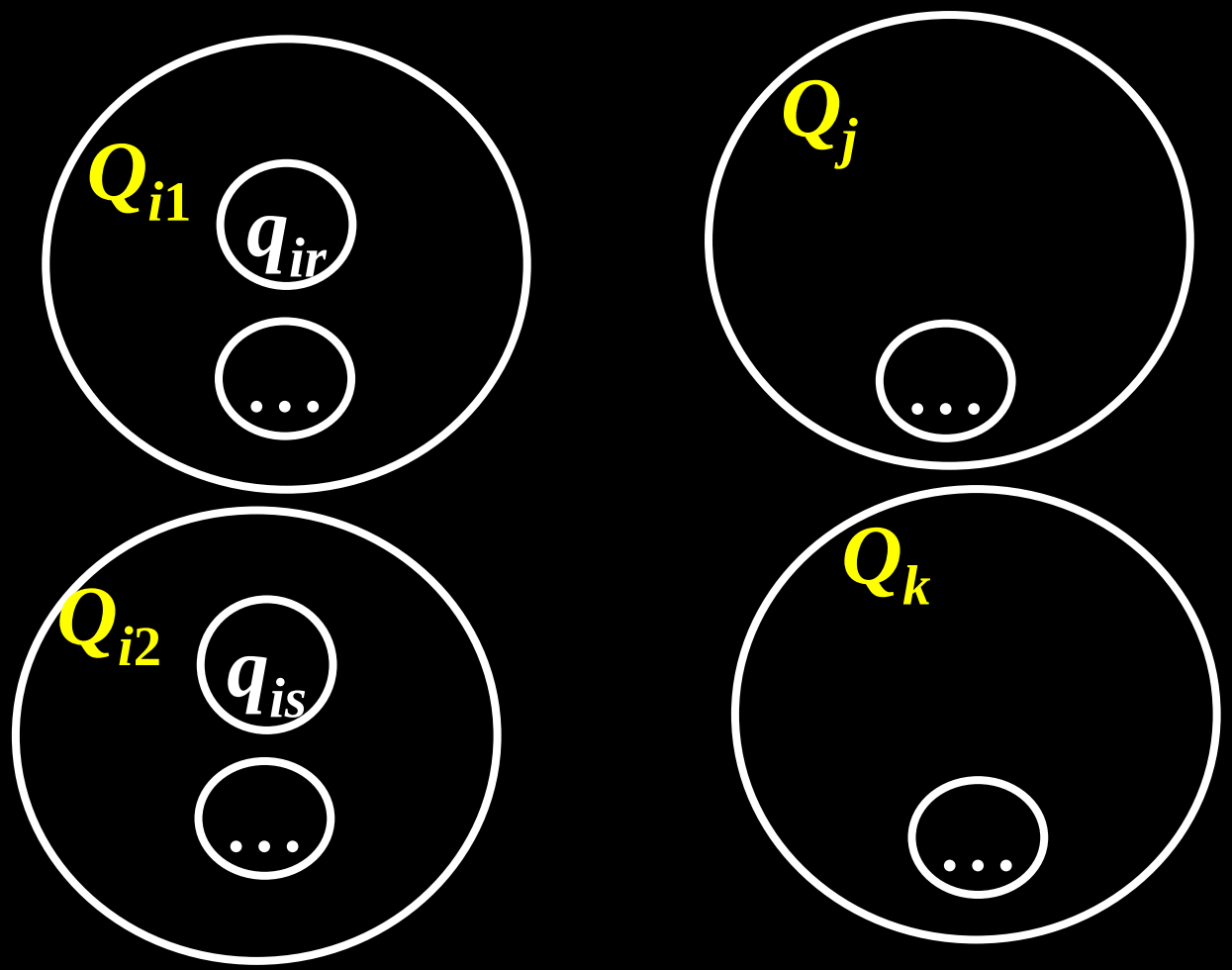
(3) 若$\pi_{new} \neq \pi$，则将其作为π再重复(2)中的过程，如此下去，直到最后得到一个划分π，使$\pi_{new} = \pi$，即π中的各个子集不能再进行划分为止。

(4)所得的最后划分π，对它的每个子集$Q_j = \{q_{j1}, q_{j2}, \cdots, q_{jr}\}$进行重新命名为一个状态，如$p_j$作为$\pi$ 中子集Q_j的名字，这些新命名的状态p_j组成了M'的状态集Q'。并且，若Q_j中含有M的初态,则p_j为M'的初态;若Q_j中含有M的终态，则p_j为M'的终态。此外，对状态转移函数作相应的修改。

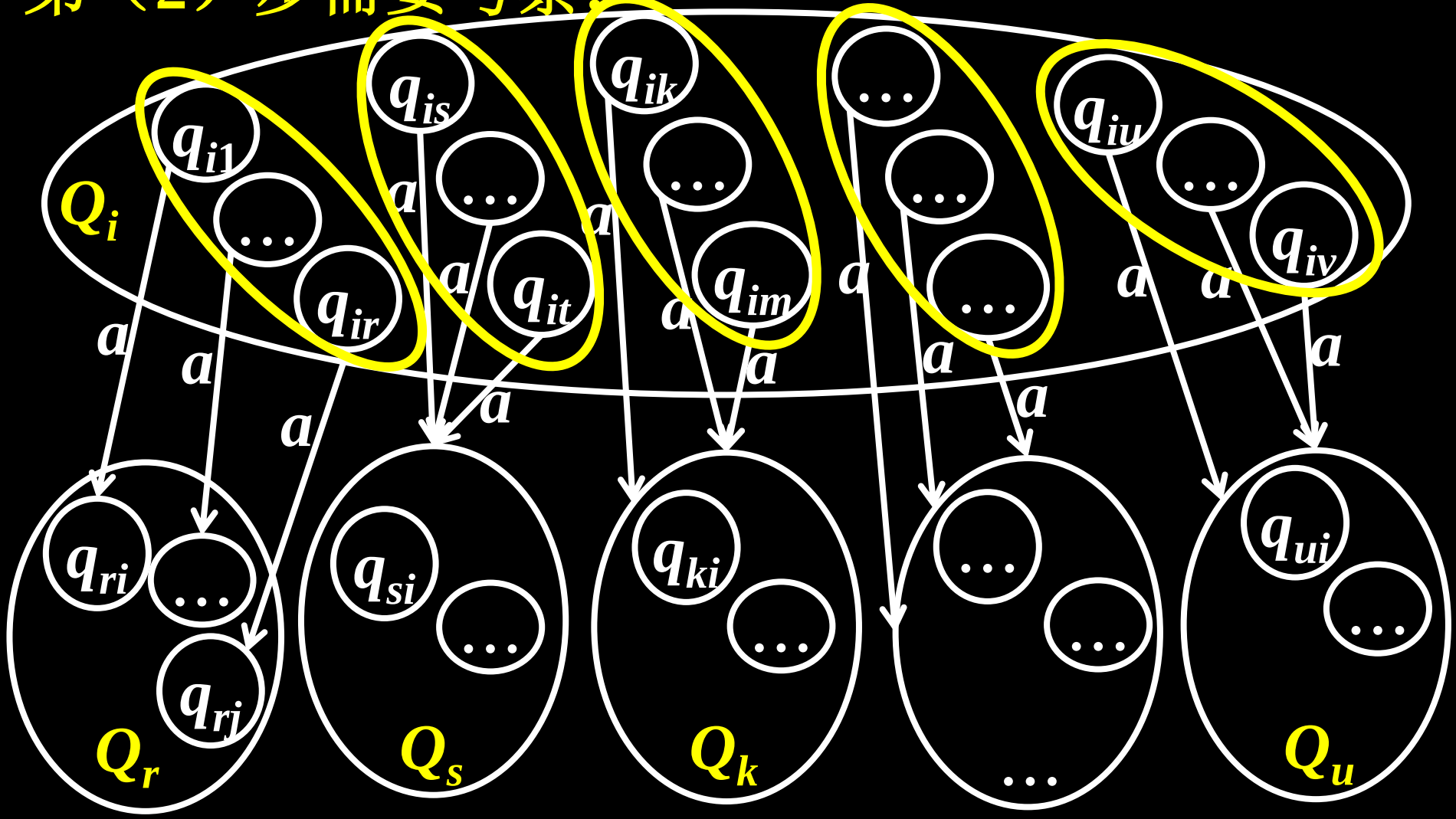
(2) 需要对每一个子集$Q_i=\{q_{i1}, q_{i2}, \ldots, q_{in}\}$中各状态$q_{ir}$ ($q_{ir}\in Q, 1\le r\le n$) 进行考察，确认是否还能对它们进行划分。



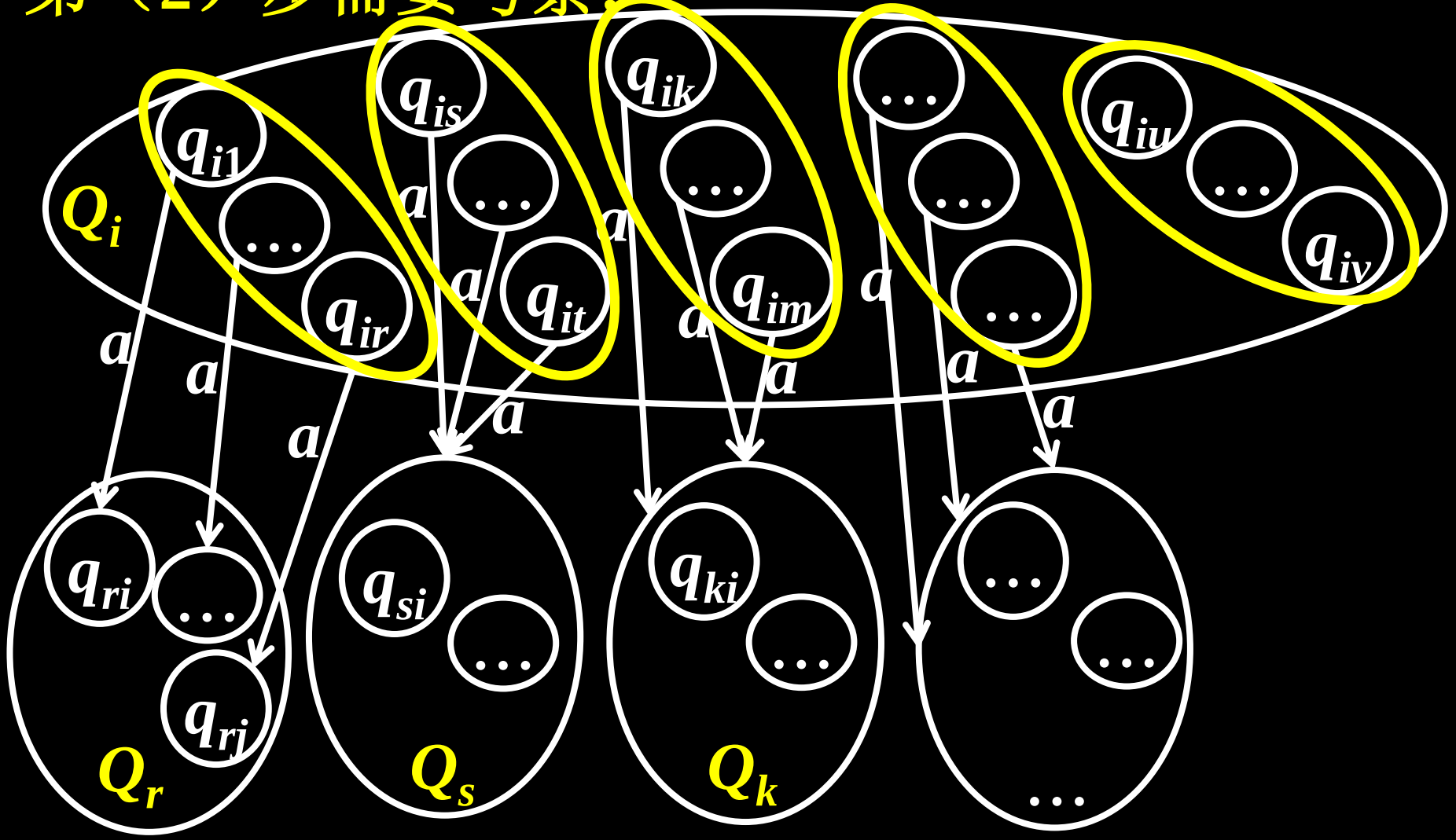
(2) 需要对每一个子集$Q_i=\{q_{i1}, q_{i2}, \ldots, q_{in}\}$中各状态$q_{ir}$ ($q_{ir}\in Q, 1\le r\le n$) 进行考察，确认是否还能对它们进行划分。



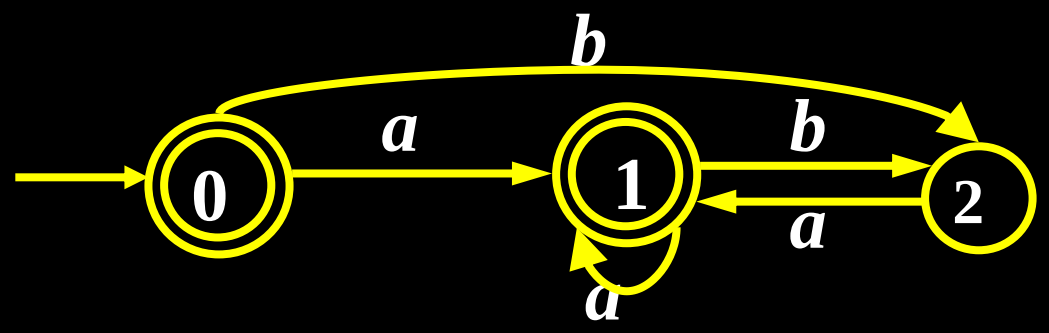
第（2）步需要考察：


Q_i
q_{i1}
q_{ir}
q_{is}
q_{it}
q_{ik}
q_{im}
q_{iu}
q_{iv}
q_{ri}
q_{rj}
q_{si}
q_{ki}
q_{ui}
Q_r
Q_s
Q_k
Q_u
a

第（2）步需要考察：



例2.30： 最小化下图的确定有限自动机DFA M'。



Step1: 形成基本划分。划分为终态集和非终态集。

$P_0 =$ （ {0,1} , {2}）

考察：

$f(0,a)=1\in\boxed{\{0,1\}}$ $f(0,b)=2\in\boxed{\{2\}}$

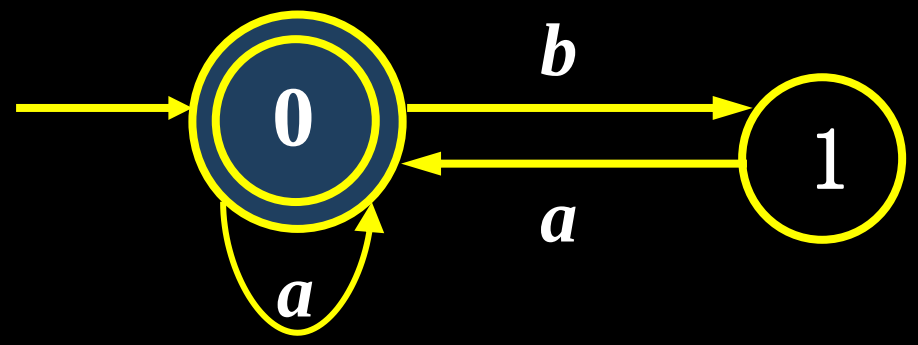
$f(1,a)=1\in\boxed{\{0,1\}}$ $f(1,b)=2\in\boxed{\{2\}}$

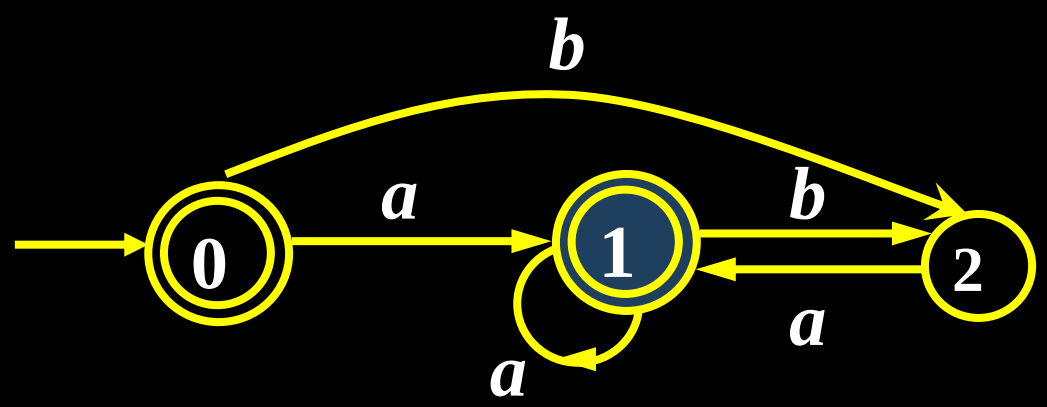
不可对{ 0,1 }再分

Step2: 重新命名。令 {0,1}为0，令{2}为1。

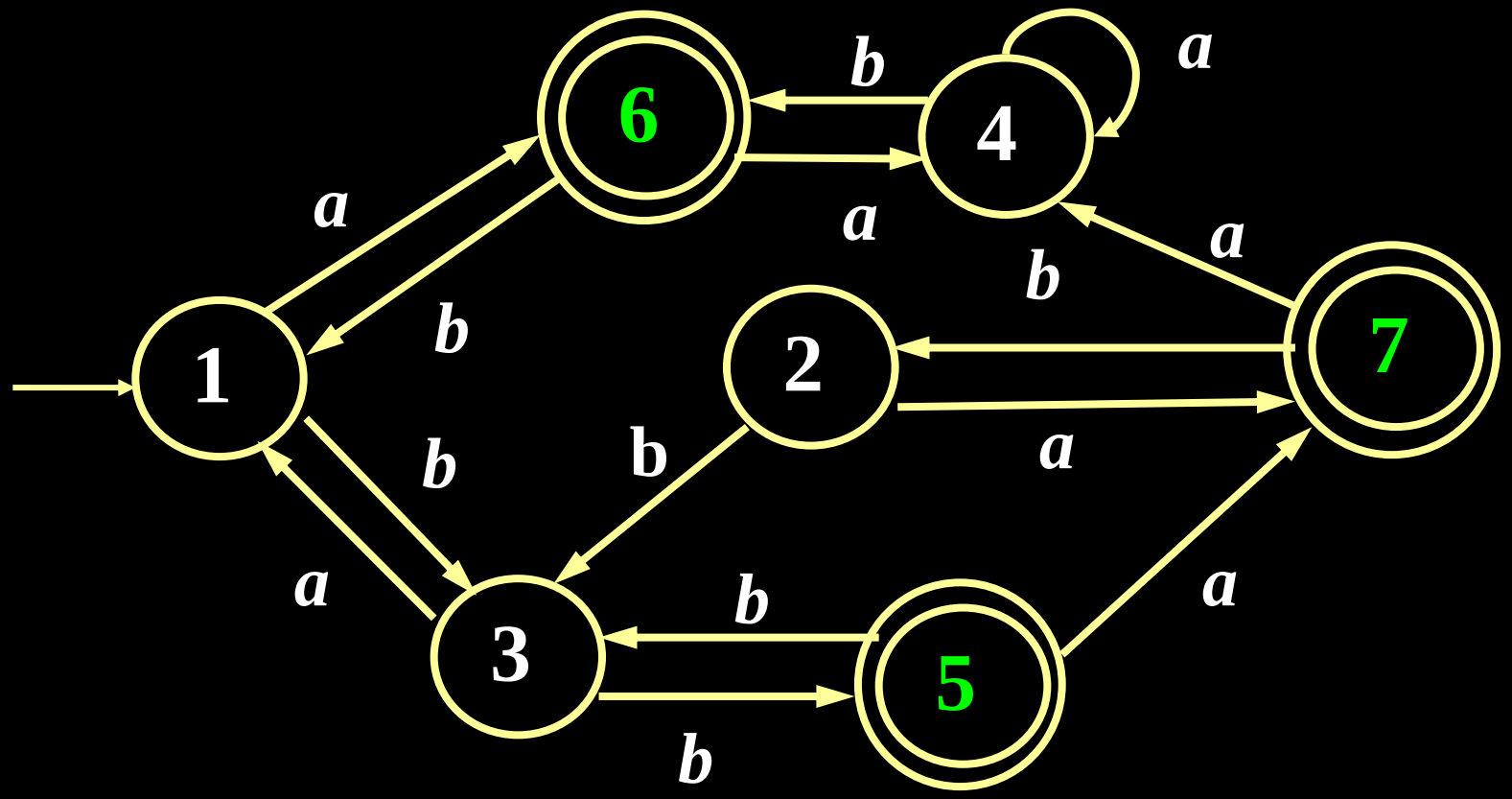
化简后的DFA *M*

重新命名：{0,1}为0，{2}为1。





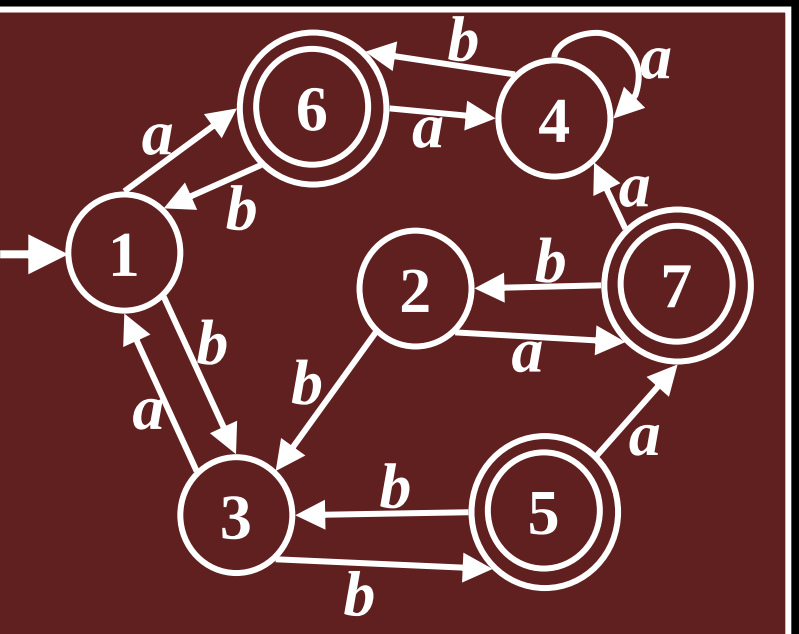
例2.31： 对下图的DFA *M*化简。



第一步，对M的状态形成基本划分π_0，π_0有两个组q_1, q_2，即

$$\pi_0 = (q_1, q_2) = (\{1, 2, 3, 4\}, \{5, 6, 7\})$$

第二步，考察 π_0 中的q_1：



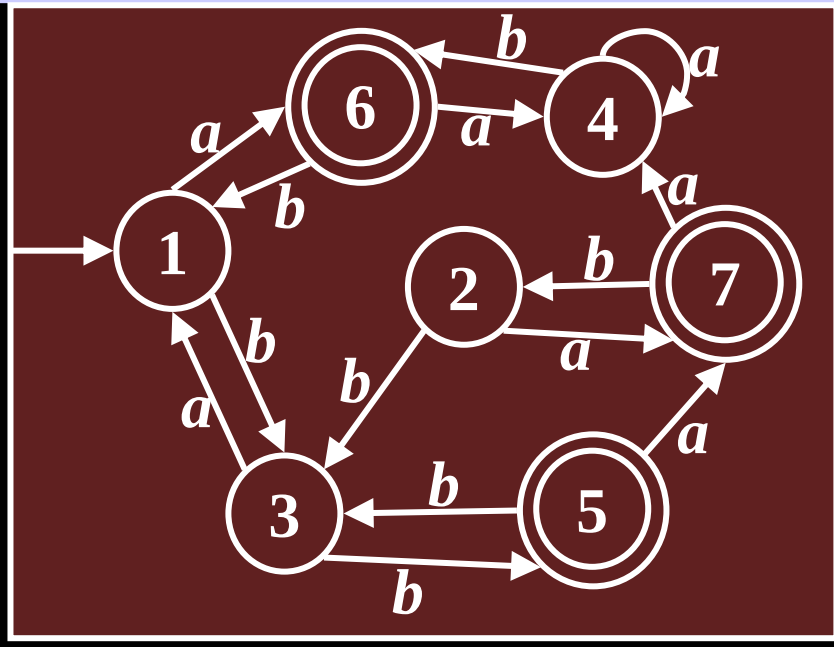
$f(1,a)=6 \in q_2$

$f(2,a)=7 \in q_2$

$f(3,a)=1 \in q_1$

$f(4,a)=4 \in q_1$

输入a的情况下，q_1中的状态1,2与状态3,4 是不等价的。



对q_1进行划分，形成：

$q_1=(q_3, q_4)=(\{1,2\}, \{3,4\})$

由此，对基本划分π_0经考察后，形成新的划分π_1：

$\pi_1 = (q_2, q_3, q_4)$

$= (\{5,6,7\}, \{1,2\}, \{3,4\})$

第三步，考察π_1中的q_2：

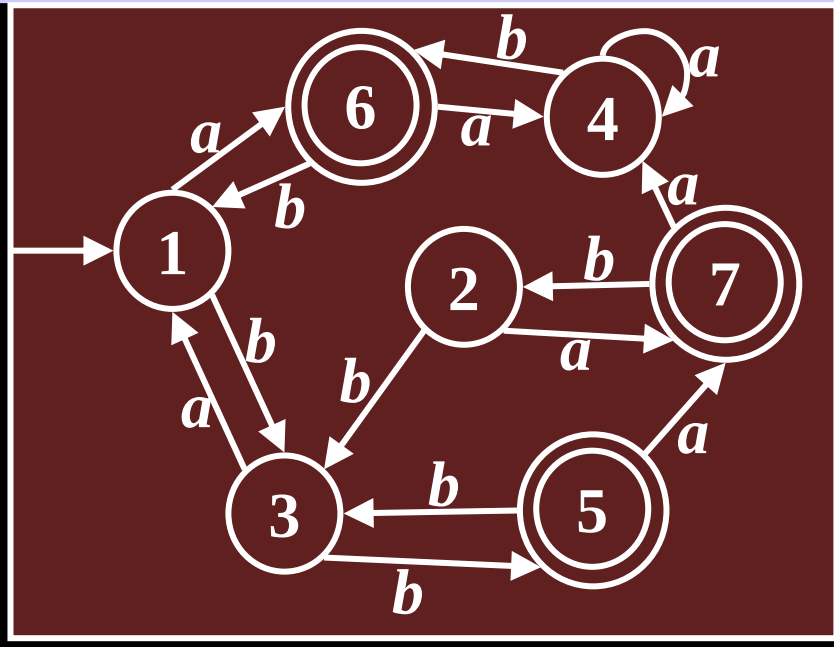
$f(5,a)=7 \in q_2$

$f(6,a)=4 \in q_4$

$f(7,a)=4 \in q_4$

输入a的情况下，

q_2中的状态5与状态6,7是不等价的。



对q_2进行划分，形成：

q_2 =(q_5, q_6)=({5}，{6, 7}）

由此，对划分π_1经考察后，

形成新的划分π_2：

π_2 =（q_3，q_4，q_5，q_6）

= ({1,2}，{3,4}，{5}，{6,7})

第四步，考察π_2中q_3，

$f(1,a)=6 \in q_6$　$f(1,b)=3 \in q_4$

$f(2,a)=7 \in q_6$　$f(2,b)=3 \in q_4$

划分不变

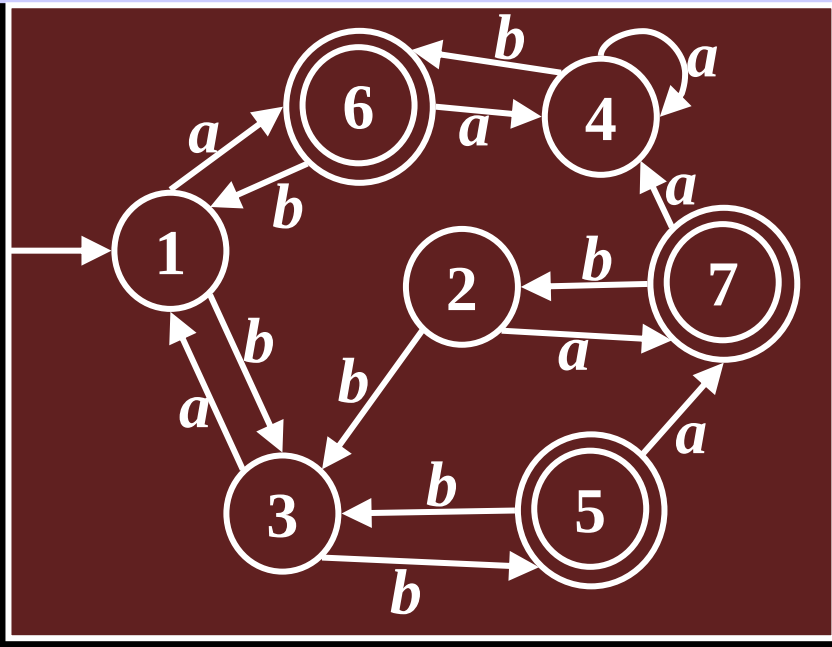
π_2 =（q_3，q_4，q_5，q_6）

=({1,2}，{3,4}，{5}，{6,7})

考察π_2中q_4

$f(3,a)=1 \in q_3$

$f(4,a)=4 \in q_4$



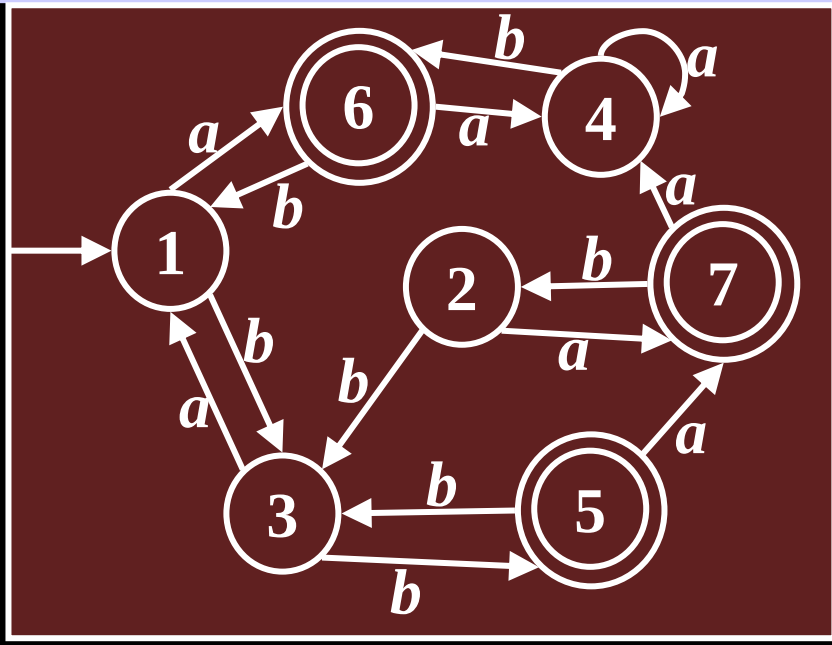
输入a的情况下，

q_4中的状态3与状态 4是不等价的。

对q_4 进行划分，形成：q_4=（q_7, q_8）=（{3}, {4}）

对划分π_2经考察后，形成新的划分π_3：

π_3 =(q_3, q_5, q_6, q_7, q_8)=({1,2}, {5}, {6,7} , {3}, {4})



第六步，考察

$\pi_3 = (q_3, q_5, q_6, q_7, q_8)$

$= (\{1,2\}, \{5\}, \{6,7\}, \{3\}, \{4\})$

中的q_3,

$f(1,a)=6 \in q_6$ $f(1,b)=3 \in q_7$

$f(2,a)=7 \in q_6$ $f(2,b)=3 \in q_7$

划分不变

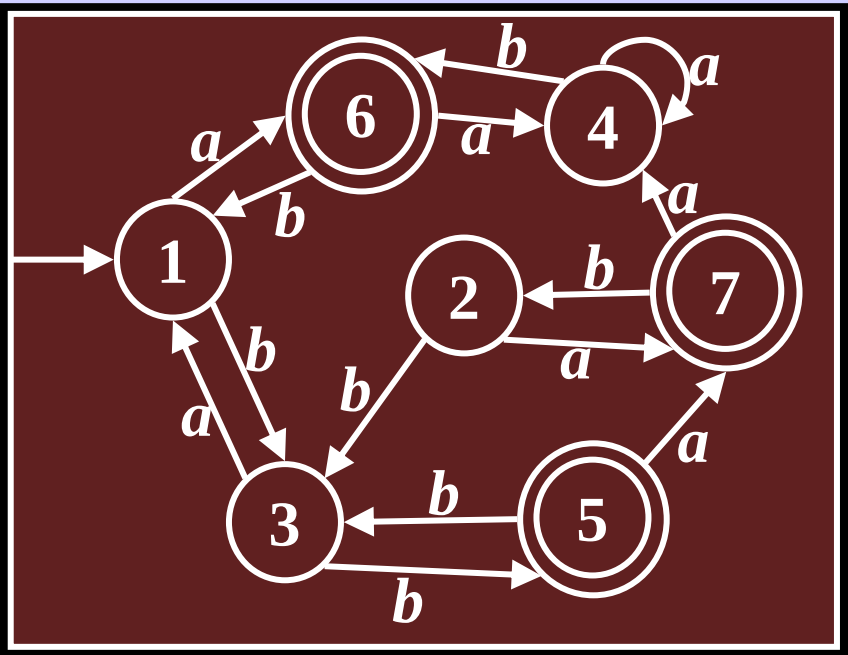
考察π_3中的q_6,

$f(6,a)=4 \in q_8$ $f(6,b)=1 \in q_3$

$f(7,a)=4 \in q_8$ $f(7,b)=2 \in q_3$

划分不变

此划分为最后划分



简化的DFA

	a	b
0	4	1
1	0	3
2	2	4
3*	4	1
4*	2	0

第七步，重新命名。

$$\pi_2 = (q_3,\ q_5, q_6, q_7, q_8)$$

$$= (\underbrace{\{1,2\}}_{0}, \underbrace{\{5\}}_{3}, \underbrace{\{6,7\}}_{4}, \underbrace{\{3\}}_{1}, \underbrace{\{4\}}_{2})$$

第二章第二节提要

2.2.1 DFA

- 定义及三种表示方法
- 识别机制：语言与DFA转换

2.2.2 NFA

- 定义及三种表示法
- 识别机制：语言与NFA转换

2.2.3 DFA与NFA的等价

- ε闭包，a弧转移
- NFA的确定化—子集法

2.2.4 DFA的化简

- 消除无关状态
- 消除等价状态—划分法

✌ 第二章第二节

end